Automated E-Learning Data Pipeline

Cloud Solution — Technical Documentation

Microsoft Fabric • dbt • Power BI

Repository: Mahmoud-E-28/Automated_E-Learning_Data_Pipeline
Document Version 1.0

Table of Contents

1. Project Overview

This pipeline aggregates online course catalog data from three e-learning platforms — Coursera, Udemy, and Udacity — into a single, cleaned, and standardized analytical data warehouse. The output is a Kimball-style star schema designed to support BI reporting in Power BI and to act as a high-quality feature source for downstream machine learning systems such as course recommendation engines.

The cloud solution is built on Microsoft Fabric and runs the full extract, transform, and load (ETL) cycle as an orchestrated Fabric Data Pipeline, with dbt performing all SQL-based transformation logic against a Fabric Warehouse/Lakehouse and Power BI consuming the resulting semantic model.

1.1 Goals

- Unify inconsistent schemas, naming conventions, and units across three independent course platforms.
- Apply consistent business rules for level, language, price, and duration so cross-platform comparisons are valid.
- Track historical changes to course attributes using Slowly Changing Dimensions (SCD Type 2).
- Enforce data quality through automated dbt tests on every build.
- Publish a refreshed Power BI semantic model after every successful transformation run.

1.2 Source Platforms

Platform	Raw Source Table	Notes
Coursera	coursera_final_data	Scraped catalog data; course_id used as natural key
Udemy	udemy_final_data	Filters out rows where course_id is not numeric
Udacity	udacity_final_data	Monthly subscription pricing model

2. System Architecture

The cloud solution runs entirely inside a Microsoft Fabric workspace. A single orchestrated Fabric Data Pipeline ('Pipelining') sequences three stages: data ingestion via a Fabric Notebook, transformation via an embedded dbt job, and a Power BI semantic model refresh.

2.1 Pipeline Flow

1. **Notebook1 (TridentNotebook)** — Loads/refreshes raw course data into the Fabric Lakehouse/Warehouse tables (coursera_final_data, udemy_final_data, udacity_final_data) under the dbo schema. This stage represents Extract & Load.
2. **dbt job1 (InvokeDataBuildToolJob)** — Triggered only on success of Notebook1. Runs the dbt project ('build' operation, 4 threads) which executes staging, intermediate, snapshot, and mart models in dependency order, including all data quality tests.
3. **Semantic model refresh1 (PBISemanticModelRefresh)** — Triggered only on success of dbt job1. Issues a transactional refresh of the Power BI semantic model, refreshing all nine warehouse tables (dim_date, dim_instructor, dim_language, dim_level, dim_offering_type, dim_platform, dim_offering, dim_domain, fact_offerings) and waits for completion before the pipeline run is marked successful.

2.2 Architecture Diagram (Logical)

```
Coursera / Udemy / Udacity (raw extracts)
            |
            v
   [Notebook1 - Fabric Notebook]
   Loads raw_data.dbo.*_final_data
            |  (on success)
            v
   [dbt job1 - InvokeDataBuildToolJob]
   staging -> intermediate -> snapshot -> marts (+ tests)
            |  (on success)
            v
   [Semantic model refresh1 - PBISemanticModelRefresh]
   Refreshes Power BI dataset (9 tables, transactional)
            |
            v
        Power BI Dashboard (PowerBI_Dashboard.pbix)
```

2.3 Technology Stack

Layer	Technology	Purpose
Orchestration	Microsoft Fabric Data Pipeline	Sequences notebook, dbt, and Power BI refresh activities
Ingestion	Fabric Notebook (Trident)	Loads raw platform data into the warehouse
Transformation	dbt (data build tool)	SQL-based modeling, testing, and snapshotting
Storage	Fabric Warehouse (T-SQL dialect)	Hosts raw, staging, intermediate, and mart tables

Layer	Technology	Purpose
Dependency Management	dbt-labs/dbt_utils v1.4.0	Provides reusable test macros (e.g. accepted_range)
Reporting	Power BI (PowerBI_Dashboard.pbix)	Semantic model and dashboard layer

3. dbt Project Structure

The transformation logic lives entirely in the 'cloud solution' directory and follows a layered dbt architecture: staging, intermediate, snapshots, and marts.

3.1 Directory Layout

```
cloud solution/
├── dbt_project.yml
├── packages.yml
├── Pipelining/
│   ├── Pipelining .json        (Fabric pipeline definition)
│   └── manifest.json           (dbt compiled manifest)
├── macros/
│   ├── parse_duration.sql
│   └── standardize_language.sql
├── models/
│   ├── staging/
│   │   ├── sources.yml
│   │   ├── schema.yml
│   │   ├── stg_coursera.sql
│   │   ├── stg_udemy.sql
│   │   └── stg_udacity.sql
│   ├── intermediate/
│   │   ├── schema.yml
│   │   ├── int_courses_combined.sql
│   │   └── int_courses_standardized.sql
│   └── marts/
│       ├── schema.yml
│       ├── dimensions/
│       │   ├── dim_date.sql
│       │   ├── dim_domain.sql
│       │   ├── dim_instructor.sql
│       │   ├── dim_language.sql
│       │   ├── dim_level.sql
│       │   ├── dim_offering.sql
│       │   ├── dim_offering_type.sql
│       │   └── dim_platform.sql
│       └── facts/
│           └── fact_offerings.sql
├── snapshots/
│   └── offering_snapshot.sql
└── PowerBI_Dashboard.pbix
```

3.2 dbt_project.yml Configuration

Layer	Materialization
staging	view
intermediate	view
marts	table

Project name: my_dbt_project. Profile: default. Standard dbt paths are used for models, seeds, snapshots, macros, and tests.

3.3 Package Dependencies

```
packages:
  - package: dbt-labs/dbt_utils
    version: 1.4.0
```

dbt_utils supplies the accepted_range and unique_combination_of_columns generic tests used throughout the schema files.

4. Staging Layer

Staging models read directly from the raw source tables and perform light cleaning: type casting, trimming whitespace, deduplication, and column renaming to a shared naming convention. Each model is materialized as a view.

4.1 Sources

Defined in models/staging/sources.yml under the raw_data source (schema: dbo):

- coursera_final_data
- udemy_final_data
- udacity_final_data

4.2 stg_coursera

Casts course_id to varchar, trims title and description, hardcodes platform = 'Coursera', applies the standardize_language and parse_duration macros, casts numeric fields (rating, price, review_count, enrolled_students), and deduplicates using ROW_NUMBER() partitioned by course_id ordered by last_updated descending (keeping rn = 1). Rows with a null course_id are excluded.

4.3 stg_udemy

Follows the same pattern as stg_coursera, with platform = 'Udemy'. Source-specific column names differ (e.g. No_of_Reviews, rate, offering_Type) and are mapped to the shared output schema. Rows are additionally filtered to require that course_id can be cast to an integer (TRY_CAST(course_id as int) is not null), guarding against malformed scraped identifiers.

4.4 stg_udacity

Follows the same pattern, with platform = 'Udacity'. Notable differences: duration is sourced from Duration_Hours rather than a free-text Duration field, price is sourced from monthly_price (reflecting Udacity's subscription model), and domain is derived from Best_Category.

4.5 Shared Output Schema (all staging models)

Column	Type	Description
course_id	varchar(100)	Natural key from the source platform
title	string	Course title, trimmed
course_url	string	Link to the course
platform	string	Literal: Coursera / Udemy / Udacity
language	string	Standardized via standardize_language macro
description	string	Trimmed course description
skills	string	Trimmed/normalized skills list
level	string	Raw difficulty level (standardized downstream)
instructor	string	Trimmed instructor name
domain_name	string	Subject/category
offering_type	varchar(100)	Format of the offering
last_updated	date	Parsed via TRY_CONVERT / TRY_CAST
duration_raw	string	Original duration text/value
duration_hours	float	Parsed numeric hours via parse_duration macro
review_count	int	Number of reviews
rating	float	Average rating
price	float	Listed price
price_model	string	One Time / Subscription / etc.
enrolled_students	int	Enrollment count

4.6 Staging Tests (schema.yml)

Applied identically to stg_coursera, stg_udemy, and stg_udacity:

- course_id — unique, not_null
- title — not_null
- platform — not_null

5. Intermediate Layer

5.1 int_courses_combined

Performs a UNION ALL of stg_coursera, stg_udacity, and stg_udemy into a single combined view, preserving the shared staging schema. This is the first point where data from all three platforms exists in one table.

Tested for: course_id not_null; platform not_null and restricted to the accepted_values ['Coursera', 'Udacity', 'Udemy']; title not_null; and a composite uniqueness test (dbt_utils.unique_combination_of_columns) on (course_id, platform), since course_id alone is not unique across platforms.

5.2 int_courses_standardized

Builds on int_courses_combined and applies the core business logic of the pipeline, producing the analytical fields consumed by the fact table. Key transformations:

Derived Field	Logic
level	Maps free-text variants (e.g. 'beginner level', 'introductory') to one of: Beginner, Intermediate, Advanced, All Levels, Unknown
rating	Out-of-range values (< 0 or > 5) are nulled out to maintain data quality
price_model	Normalized to Subscription, One Time, or Unknown
price_category	Bucketed from price: Free (0), Cheap (<500), Medium (<2000), Expensive (<5000), Premium (>=5000), Unknown (null)
duration_category	Bucketed from duration_hours: Short (<5h), Medium (<20h), Long (<50h), Extensive (>=50h), Unknown (null)
popularity_score	ROUND(rating * LOG(enrolled_students + 1), 2)
engagement_rate_pct	ROUND(review_count / NULLIF(enrolled_students,0) * 100, 2)
weighted_rating	ROUND(rating * LOG(review_count + 1), 2)
cost_per_hour	ROUND(price / NULLIF(duration_hours,0), 2)
estimated_total_cost	For subscriptions: price * CEILING(duration_hours / 40.0); otherwise the listed price
value_score	ROUND(popularity_score / estimated_total_cost, 2) when estimated_total_cost > 0, else null

Tested for: course_id not_null; platform not_null/accepted_values; level restricted to {Beginner, Intermediate, Advanced, All Levels, Unknown}; price_category restricted to {Free, Cheap, Medium, Expensive, Premium, Unknown}; duration_category restricted to {Short, Medium, Long, Extensive, Unknown}; rating range-checked between 0 and 5 (severity: warn); and the same composite (course_id, platform) uniqueness test as the combined model.

6. Snapshots (SCD Type 2)

offering_snapshot.sql implements Slowly Changing Dimension Type 2 tracking on top of int_courses_standardized. This preserves the full history of how a course's descriptive content evolves over time, which is essential for recommendation systems that should not silently lose the context behind a past recommendation.

Setting	Value
target_schema	snapshots
unique_key	course_id
strategy	check
check_cols	title, description, skills, course_url

Whenever title, description, skills, or course_url changes for a given course_id, dbt closes out the previous snapshot row (setting dbt_valid_to) and inserts a new row with dbt_valid_from set to the current run time. Rows where dbt_valid_to is null represent the current version of the record.

7. Marts Layer — Star Schema

The marts layer materializes the final dimensional model as physical tables, optimized for BI consumption and ML feature extraction. It consists of eight dimension tables and one central fact table.

7.1 Dimension Tables

All dimension tables follow a common pattern: a deduplicated list of distinct values from int_courses_standardized, surrogate-keyed via ROW_NUMBER(), with a reserved key of -1 representing 'Unknown' to support safe left joins from the fact table.

Dimension	Grain	Notes
dim_platform	One row per platform	Coursera / Udemy / Udacity
dim_domain	One row per subject domain	Derived from domain_name
dim_level	One row per difficulty level	Beginner / Intermediate / Advanced / All Levels
dim_language	One row per language	Standardized language name
dim_instructor	One row per instructor	Coalesces null to 'Unknown Instructor'
dim_offering_type	One row per offering format	Coalesces null to 'Unknown'
dim_date	One row per distinct last_updated date	date_id format YYYYMMDD; includes a -1 unknown-date row
dim_offering	One row per (course, validity period)	Built from offering_snapshot; carries SCD2 effective_from/effective_to and is_current flag

7.1.1 dim_offering — SCD2 Detail

dim_offering is the only dimension sourced from the snapshot rather than directly from int_courses_standardized. Its surrogate key, offering_sk, is generated via ROW_NUMBER() ordered by (course_id, dbt_valid_from), allowing the same course to have multiple historical rows. is_current is set to 1 when dbt_valid_to is null (the live version) and 0 otherwise.

7.2 Fact Table — fact_offerings

fact_offerings is the central table connecting all dimensions. It is built from int_courses_standardized and left-joined to every dimension; any unmatched join falls back to the dimension's -1 'Unknown' key via COALESCE, guaranteeing referential integrity even with incomplete source data.

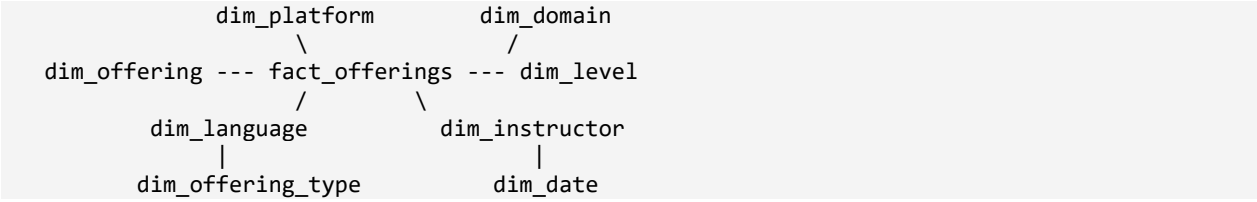
7.2.1 Foreign Keys

- offering_sk → dim_offering (joined on course_id, restricted to is_current = 1)
- platform_id → dim_platform
- domain_id → dim_domain
- level_id → dim_level
- language_id → dim_language
- instructor_id → dim_instructor
- offering_type_id → dim_offering_type
- date_id → dim_date (joined on the YYYYMMDD-formatted last_updated)

7.2.2 Measures

Measure	Description
price	Listed price
rating	Average rating (0–5, validated)
review_count	Number of reviews
enrolled_students	Enrollment count
duration_hours	Parsed numeric duration
price_category	Free / Cheap / Medium / Expensive / Premium / Unknown
duration_category	Short / Medium / Long / Extensive / Unknown
popularity_score	Calculated in the intermediate layer
engagement_rate_pct	Calculated in the intermediate layer
weighted_rating	Calculated in the intermediate layer
cost_per_hour	Calculated in the intermediate layer
estimated_total_cost	Calculated in the intermediate layer
value_score	Calculated in the intermediate layer

7.3 Entity-Relationship Overview



8. Testing & Data Quality

Data quality is enforced through dbt's generic and package-provided tests at every layer. The dbt job activity in the Fabric pipeline runs a full build (model execution plus tests) on each invocation, so a broken contract halts the pipeline before it can reach Power BI.

Layer	Test	Applies To
Staging	unique, not_null	course_id on stg_coursera / stg_udemy / stg_udacity
Staging	not_null	title, platform on all staging models
Intermediate	accepted_values	platform restricted to Coursera/Udacity/Udemy
Intermediate	accepted_values	level, price_category, duration_category enumerations
Intermediate	dbt_utils.accepted_range	rating between 0 and 5 (warn severity)
Intermediate	dbt_utils.unique_combination_of_columns	(course_id, platform) composite uniqueness
Marts	unique, not_null	dim_offering.offering_sk
Marts	relationships	Every foreign key on fact_offerings validated against its dimension

Note: rating range validation uses severity: warn rather than error, since out-of-range source ratings are nulled during transformation rather than causing the build to fail — this keeps malformed source rows from blocking the entire pipeline while still surfacing the issue for review.

9. Reusable Macros

9.1 parse_duration(column_name)

Converts heterogeneous duration text into a single float representing hours. Logic order:

4. Returns null for null or empty input.
5. If the text contains 'minute' or 'min', extracts the leading numeric portion and divides by 60.
6. If the text contains 'hour' or 'hr', extracts the leading numeric portion as-is.
7. If the value is purely numeric, casts it directly to float.
8. Otherwise returns null.

Used in all three staging models to populate duration_hours from each platform's differently-formatted duration field.

9.2 standardize_language(column_name)

A large CASE expression mapping language names and ISO-style locale codes (e.g. 'en-us', 'es-mx', 'pt-br') to a canonical, human-readable language name. Covers 29 languages including English, Spanish, Portuguese, French, German, Arabic, Chinese, Japanese, Korean, and others, with any unmatched value falling back to 'Other / Unknown'. Used identically in all three staging models.

10. Fabric Pipeline Configuration

The pipeline is defined as an ARM-style deployment template at cloud solution/Pipelining/Pipelining .json. It declares a single pipeline resource named 'Pipelining' with three sequential activities.

10.1 Activity Details

Activity	Type	Depends On	Key Settings
Notebook1	TridentNotebook	(none)	Timeout 12h, no retry; runs a Fabric notebook by notebookId/workspaceId
dbt job1	InvokeDataBuildToolJob	Notebook1 (Succeeded)	Operation: build, Threads: 4, references a Fabric dbt project job
Semantic model refresh1	PBISemanticModelRefresh	dbt job1 (Succeeded)	Transactional refresh, waitOnCompletion: true, refreshes 9 named tables

Each activity uses a 'Succeeded' dependency condition, meaning the pipeline is strictly sequential and fails fast: if ingestion fails, dbt never runs; if dbt (including its tests) fails, the Power BI dataset is never refreshed with potentially inconsistent data.

10.2 Parameters

The template exposes two connection parameters that must be supplied at deployment/runtime:

- DataBuildToolJob ma30501280104594 — the Fabric connection used to invoke the dbt job.
- PowerBIDatasets ma30501280104594 — the Fabric connection used to trigger the Power BI semantic model refresh.

11. Power BI Dashboard

PowerBI_Dashboard.pbix is the reporting layer connected to the marts (star schema) tables. After every successful dbt build, the Fabric pipeline triggers a transactional refresh of this semantic model so dashboard consumers always see data consistent with the latest validated warehouse state.

The semantic model refresh explicitly targets the following tables: dim_date, dim_instructor, dim_language, dim_level, dim_offering_type, dim_platform, dim_offering, dim_domain, and fact_offerings — i.e., the complete star schema produced by the marts layer.

12. Deployment & Setup Guide

12.1 Prerequisites

- A Microsoft Fabric workspace with a Lakehouse or Warehouse provisioned to hold the raw_data.dbo source tables.
- A Fabric-hosted dbt project (or equivalent dbt Cloud / dbt Core connection) with access to install dbt-labs/dbt_utils 1.4.0.
- A Power BI workspace and dataset corresponding to the IDs referenced in the pipeline JSON (groupId, datasetId).
- Connections registered in Fabric for: the dbt job invocation, and the Power BI semantic model refresh.

12.2 Setup Steps

9. Provision the raw tables coursera_final_data, udemy_final_data, and udacity_final_data in the dbo schema of the target warehouse — these are the inputs the staging models read from.
10. Deploy the dbt project ('cloud solution') to the Fabric dbt job, ensuring dbt_project.yml and packages.yml are included, then run dbt deps to install dbt_utils.
11. Import the ARM template at cloud solution/Pipelining/Pipelining .json into the Fabric workspace, or recreate the three activities (Notebook → dbt job → Semantic model refresh) manually with the dependency chain described in Section 10.
12. Supply the two connection parameters (DataBuildToolJob and PowerBIDatasets) for your workspace.
13. Publish or upload PowerBI_Dashboard.pbix and connect it to the marts tables in the warehouse, matching the groupId/datasetId referenced by the pipeline.
14. Trigger the pipeline manually once to validate the end-to-end run, then configure a schedule trigger in Fabric if recurring refreshes are required.

12.3 Running Locally (for development)

Within the 'cloud solution' directory, standard dbt commands apply once a profiles.yml is configured against the target Fabric Warehouse / SQL endpoint:

```
dbt deps
dbt build          # runs models + snapshots + tests in dependency order
dbt docs generate  # optional: generates browsable lineage docs
```

dbt build is preferred over running dbt run and dbt test separately, because offering_snapshot depends on int_courses_standardized and dim_offering depends on the snapshot — dbt build correctly sequences staging, snapshotting, and mart construction while testing everything in a single pass.

13. Operational Notes & Known Considerations

- Fail-fast design: because each Fabric activity depends on the prior one succeeding, a test failure inside the dbt build will prevent the Power BI dataset from refreshing with bad or incomplete data.
- Surrogate keys: every dimension reserves -1 for 'Unknown', and the fact table always resolves to a valid foreign key via COALESCE — there are no null foreign keys in fact_offerings by design.
- Course identity across platforms: course_id is only unique per platform, not globally; all uniqueness constraints in staging/intermediate use the (course_id, platform) composite key.
- Historical accuracy: because dim_offering is sourced from offering_snapshot, multiple historical rows can exist for the same course_id. Fact table joins to dim_offering filter to is_current = 1, so fact_offerings always reflects current course metadata, while full history remains queryable in the snapshot table directly.
- Subscription cost normalization: estimated_total_cost approximates the total cost of subscription-based courses (e.g. Udacity) by multiplying the price by the number of 40-hour billing periods implied by duration_hours, making cost comparable to one-time-purchase courses.
- Rating bounds: ratings outside the valid 0–5 range are set to null rather than dropped, preserving the row while preventing skewed downstream metrics (popularity_score, weighted_rating, value_score).

End of document.